MBEYA UNIVERSITY OF SCIENCE AND TECHNOLOGY

COLLEGE: INFORMATION AND COMMUNICATION TECHNOLOGY

DEPARTMENT: COMPUTER SCIENCE AND ENGINEERING

PROGRAM: BACHELOR OF COMPUTER SCIENCE

COURSE NAME: APPLICATION DEVELOPMENT FOR MOBILE DEVICES

COURSE CODE: CS 8306

LEVEL: UQF 8 (THIRD YEAR)

LECTURER'S NAME: MR. AMAN RUKOIJO

TASK TYPE: GROUP ASSIGNEMENT

PROJECT NAME: **FARM DATA MANAGEMENT SYSTEM**

S/N	NAME OF STUDENT	REGISTRATION NO.	SIGNATURE
1.	TENDAWEMA FOKAS KWIGEZA	23100533350087	T.F. Kwigeza

Table of Contents

1. Introduction & Problem Definition 3
1.1 Problem Statement 3
1.2 Motivation 3
1.3 Objectives 3
2. System Requirements 4
2.1 Functional Requirements 4
2.2 Non-Functional Requirements 5
3. System Design & Architecture 6
3.1 System Architecture Overview 6
3.2 Database Schema 7
3.3 API Architecture 7
3.4 Authentication Flow 8
4. Security & Authentication 9
4.1 Password Security 9
4.2 JWT Token Handling 9
4.3 Route Protection 9
4.4 CORS Configuration 9
4.5 Input Validation 9
5. Results, Screenshots & Conclusion 10
5.1 Application Features Implemented 10
5.2 Challenges Faced 10
5.3 Conclusion 10
5.4 Future Work 11

1. Introduction & Problem Definition

1.1 Problem Statement

Modern farming operations generate large volumes of data across multiple domains: livestock health and breeding, crop cycles and field management, financial transactions, and workforce records. Most smallholder and mid-scale farmers in developing regions rely on paper-based or no record-keeping systems, leading to poor decision-making, financial mismanagement, and reduced farm productivity.

The Farm Data Management System (FDMS) addresses this challenge by providing a centralized, digital platform that enables farmers and farm managers to capture, manage, and analyze all farm-related data from any device.

1.2 Motivation

- High data loss due to paper-based record keeping
- Difficulty in tracking livestock health and vaccination schedules
- No real-time visibility into farm income vs. expenses
- Inability to track worker performance and payroll efficiently
- Lack of accessible, mobile-friendly farm management tools in the region

1.3 Objectives

- Build a secure, role-based authentication system for farm managers
- Implement full CRUD operations for livestock, crop, finance, and worker modules
- Provide a visual dashboard with charts and key performance indicators
- Ensure the application is fully responsive for mobile and desktop use
- Deploy the application to a publicly accessible cloud environment

2. System Requirements

2.1 Functional Requirements

The following table describes the functional requirements of the FDMS:

ID	Module	Requirement
FR-01	Authentication	Users can register with name, email and password
FR-02	Authentication	Users can login and receive a JWT session token
FR-03	Authentication	All data routes are protected and require a valid token
FR-04	Livestock	Create, view, edit, and delete livestock records
FR-05	Livestock	Filter livestock by type and health status
FR-06	Crops	Create, view, edit, and delete crop and field records
FR-07	Crops	Track planting date, expected harvest, yield, and irrigation type
FR-08	Finances	Record income and expense transactions with categories
FR-09	Finances	View total income, expenses, and net profit/loss summary
FR-10	Workers	Manage farm worker records including role, salary, and status
FR-11	Dashboard	Display charts and summary statistics for all modules

2.2 Non-Functional Requirements

Category	Requirement
Security	Passwords must be hashed using bcrypt with minimum 12 salt rounds
Security	All API endpoints must be protected with JWT middleware
Performance	API responses should return within 2 seconds under normal load
Usability	Interface must be intuitive and usable without training
Responsiveness	Application must work on screen widths from 320px to 1920px
Availability	Application must be hosted and publicly accessible 24/7
Scalability	Database must support growth with proper indexing and schema design
Maintainability	Code must be modular, well-structured, and commented

3. System Design & Architecture

3.1 System Architecture Overview

The FDMS follows a three-tier client-server architecture consisting of a React frontend (Presentation Layer), a Node.js/Express backend API (Application Layer), and a MongoDB Atlas database (Data Layer). Communication between layers uses RESTful HTTP APIs with JSON payloads.

Layer	Technology	Hosting	Role
Frontend (Client)	React 18 + Vite	Vercel	User interface and state management
Backend (API)	Node.js + Express.js	Render	Business logic, routing, authentication
Database	MongoDB Atlas	MongoDB Cloud	Data persistence and querying

3.2 Database Schema

The system uses 5 MongoDB collections with the following structures:

Collection	Key Fields	Relationships
users	name, email, password (hashed), role	Authenticates all API access
livestocks	name, type, breed, gender, age, weight, healthStatus, tagNumber, vaccinations[]	Independent collection
crops	name, type, fieldName, fieldSize, plantingDate, expectedHarvestDate, status, yieldAmount	Independent collection
finances	type, category, amount, description, date, paymentMethod, relatedTo	Can reference livestock/crop by name
workers	firstName, lastName, phone, role, employmentType, salary, status, startDate	Independent collection

3.3 API Architecture

The backend exposes a RESTful API following standard HTTP conventions:

- GET /api/{resource} – Retrieve all records (with optional query filters)
- GET /api/{resource}/:id – Retrieve a single record by ID
- POST /api/{resource} – Create a new record
- PUT /api/{resource}/:id – Update an existing record
- DELETE /api/{resource}/:id – Delete a record
- GET /api/{resource}/stats/summary – Aggregated statistics

3.4 Authentication Flow

The system uses stateless JWT authentication:

- User submits email and password via POST /api/auth/login
- Backend verifies credentials, compares hashed password using bcrypt
- On success, backend signs and returns a JWT token (7-day expiry)
- Frontend stores token in localStorage and attaches it to all requests via Authorization: Bearer header
- Auth middleware validates the token on every protected route before processing the request

4. Security & Authentication

4.1 Password Security

User passwords are never stored in plain text. The bcryptjs library is used to hash passwords with 12 salt rounds before storing in MongoDB. During login, the candidate password is compared against the stored hash using bcrypt. compare(), ensuring timing-safe comparison.

4.2 JWT Token Handling

JSON Web Tokens (JWT) are used for stateless session management. Tokens are signed using a secret key defined in the server environment variable JWT_SECRET. Each token encodes the user's ID and email, and expires after 7 days. The frontend attaches tokens via the Authorization HTTP header.

4.3 Route Protection

All data endpoints (livestock, crops, finances, workers) are protected by the auth middleware function. This middleware extracts and verifies the JWT from the request header. If the token is missing or invalid, a 401 Unauthorized response is returned immediately, preventing any data access.

4.4 CORS Configuration

Cross-Origin Resource Sharing (CORS) is configured on the backend to accept requests only from the trusted frontend origin defined in the FRONTEND_URL environment variable, preventing unauthorized cross-origin requests.

4.5 Input Validation

All incoming data is validated at the Mongoose model level using required fields, enum constraints, and min/max values. Invalid payloads return a 400 Bad Request with descriptive error messages.

5. Results, Screenshots & Conclusion

5.1 Application Features Implemented

Feature	Status	Notes
User Registration & Login	☑ Complete	JWT + bcrypt, form validation
Livestock CRUD	☑ Complete	All 4 operations + filter by type/health
Crops CRUD	☑ Complete	All 4 operations + filter by type/status
Finance CRUD	☑ Complete	Income/Expense tracking + P&L summary
Workers CRUD	☑ Complete	All 4 operations + role/status filters
Dashboard Charts	☑ Complete	Recharts pie, bar charts, stat cards
Mobile Responsive	☑ Complete	Collapsible sidebar, responsive grid
API Auth Middleware	☑ Complete	JWT protection on all data routes
Deployment	☑ Complete	Render (backend) + Vercel (frontend)

5.2 Challenges Faced

- Configuring CORS correctly between Render and Vercel deployments required careful setup of environment variables and origin whitelisting.
- Implementing the mobile-responsive sidebar with smooth animation required careful CSS work with transforms and z-index layering.
- Designing the financial summary aggregation pipeline in MongoDB required learning the $group and $match aggregation stages.

5.3 Conclusion

The Farm Data Management System successfully delivers a complete, full-stack mobile-first web application that addresses real data management needs for farm operations. All four CRUD modules are fully functional, the application is secured with JWT authentication and bcrypt password hashing, and the system is deployed to public cloud infrastructure.

5.4 Future Work

- Add multi-farm support with organization-level accounts
- Implement livestock vaccination reminder notifications via email/SMS
- Add report export functionality (PDF/Excel) for financial records
- Integrate weather API for crop planning recommendations
- Build a Progressive Web App (PWA) with offline support and push notifications
- Add role-based access control with manager vs. viewer permissions

Links

YouTube Demonstrations: https://youtu.be/Q0V7xyGWvSU

GitHub Repository: https://github.com/FOCUSJTF100/farm-management.git